

Term Vocabulary and Postings List

Academic Year 2024/2025

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Term Vocabulary

The Document

Building a IR system starts from a collection of documents to be indexed.

But how we can define documents?

In general, we can say that a document is a **sequence of text**, but before using them for indexing, there are many issues that we need to face out:

- **coding**: usually an english document is encoded in ASCII, but there are many other encoding schemes such as Unicode UTF-8 or national or vendor-specific standards
- **format**: text, XML with tags, MS Word, PDF, zip

The Document Unit

It is also important to define the **document unit**.

It is easy to think to the document unit like a single document, and this is correct when talk about files in a folder. But think about mails: many applications store several email in a single file stored in the file system. What is the document unit here? The file or the email? Or again, an email can attach several documents, or a zip file containing several documents. Again, what is the document unit here?

The Document Unit

But we need to face the same problem even when we want to index an entire book.

It is not a good idea to consider the whole book as a single document (if I'm looking for the word *italian car*, the book might have *italian* in the first chapter and *car* in the last chapter, but they are not relevant for the search). Perhaps, for such a kind of documents, the best Document Unit is a single chapter

The Document Unit

But keeping going on with this reasoning, why don't we decide to use each single paragraph like a Document Unit? So, in conclusion, defining the right Document Unit is not simple at all.

It becomes clear that there is a **precision/recall tradeoff** here.

Too small Document Unit: we miss some important information because terms are distributed over several mini-documents

Too large Document Unit: relevant information is hard to find for the user needs

The Document Unit

An IR system should be designed to offer **choices of granularity**.

The person who is deploying the system must have a good understanding of the document collection, the users, and their likely information needs and usage patterns.

Tokenization: the general rule

Once the Document Unit has been defined and the Document has been converted in a sequence of text (by decoding the text), **the document is tokenized**.

A **token** is a sequence of characters grouped together as a **useful semantic unit** for processing.

Usually, a token is separate from the others by a **white space** or by a **punctuation sign**.

Input: Friends, Romans, Countrymen, lend me your ears;

Output:

Friends	Romans	Countrymen	lend	me	your	ears
---------	--------	------------	------	----	------	------

Token, Type and Term

We should not make confusion about Token, Type and Term

- A **Token** is a sequence of characters grouped together as a **useful semantic unit** for processing.
- A **Type** is the class of all tokens containing the same character sequence
- A **Term** is a type that is included in the IR system's dictionary

Token, Type and Term

For example:

Document to be indexed

To sleep, perchance to dream

5 Tokens: *to, sleep, perchance, to, dream*

4 Types: *to, sleep, perchance, dream* (since there are 2 instances of *to*)

3 Terms: *sleep, perchance, dream* (since *to* is omitted from the index)

Tokenization

Some **cleaning operations** are done during tokenization (for example punctuation is dropped).

Input: Friends, Romans, Countrymen, lend me your ears;

Output:

Friends	Romans	Countrymen	lend	me	your	ears
---------	--------	------------	------	----	------	------

Tokenization brings along **a lot of problems**, sometimes strictly related to the language characteristics, as we see in the follow.

What are correct tokens?

The previous example of tokenization was very obvious. But defining the right tokenization is not always obvious.

For example, consider this sentence:

Mr. O'Neill thinks that the boys' stories about Chile's capital aren't amusing.

What is **the best tokenization** of *O'Neill* and *aren't*?

oneill	
o'neill	
o'	neill
o	neill?

aren't	
arent	
are	n't
aren	t?

What are correct tokens?

The simple strategy is **to split on all non-alphanumeric characters**:

o neill

aren t

But the first looks OK

The second instead looks intuitively bad.

oneill
o'neill
o' neill
o neill?

aren't
arent
are n't
aren t?

What are correct tokens?

The choice of tokenization determines which Boolean queries will match.

For example, if the user searches for the term *neill AND capital* this will match in two cases but not the other two.

oneill	
o'neill	
o'	neill
o	neill?

aren't	
arent	
are	n't
aren	t?

Language and Domain Identification

Tokenization is language-specific.

Before starting tokenizing, it is required to **identify the language**. This problem can be addressed using Classification Methods

Even the **domain** is important. Documents describing **Computer Technology** could contain specific IP address (*142.32.48.231*) which should be tokenized like as single token. Documents describing **Aircraft** might contain names like *B-52*, and again it should be considered like a single token.

Hyphenation

English language makes use of **Hyphen**.

For example: co-education, Hewlett-Packard, a six-minute activity

How those words should be considered? Only one token or two tokens?

Of course, the choice restricts what people can search for.

One heuristic can be to allow short hyphenated prefixes on words, but not longer hyphenated forms

Apostrophe

Many latin-based languages (like Italian, French, Spanish) makes use of apostrophe.

For example: *l'albero*

It would be preferred that documents containing *l'albero* and *un albero* are indexed under *albero*

White Space

Another aspect that makes tokenization harder is **the presence of white space** between words that should not be considered as two or more tokens.

For example: *Los Angeles, San Francisco, New York University*

The same happens for phone numbers (*800 2342333*) or dates (*Mar 11, 1983*)

Compound Nouns

Other languages make the problem harder in new ways.

German writes **compound names** without spaces

For example:

Lebensversicherungsgesellschaftsangestellter = Life Insurance Company Employee

Here, we can use a ***compound-splitter module***: by using a vocabulary the compound noun is splitted in words when they appear in the vocabulary

Stop Words: dropping common terms

Not all tokens should be taken into account.

Many of them, indeed, are excluded from the vocabulary (because for example they are very frequent).

Those common terms are named **Stop Words**

Which token should be ignored? The general strategy is to sort all terms by **collection frequency** (total number of times each term appears in the document collection) and drop all those that are most common

Stop List: Dropping common terms

To avoid dropping domain-related terms that are frequent in the documents, an alternative is a **Stop List**: a list of all terms to be discarded.

a	an	and	are	as	at	be	by	for	from
has	he	in	is	it	its	of	on	that	the
to	was	were	will	with					

Using a Stop List **reduces the number of postings** that the system has to store

Dropping common terms: the side effect

Sadly, dropping common terms has a **side effect**: Search results might not be very precise.

For example, suppose that we want to search for a song. Think about songs like *To be or not to be, Let It Be, I don't want to be.* Many of these tokens are common terms (*to, be, or, not, it, don't*). If we drop them off from indexing, of course **searches are not precise**.

For this reasons, Search Engines makes use of a **very small Stop List** (7-12 terms) or rather no Stop List at all.

Normalization

Tokens should also be normalized:

- **Accents and diacritics** should be removed (In many cases, users will enter queries for words without diacritics, whether for reasons of speed, laziness, limited software, or habits)
- **British vs American:** some British words are slightly different from the equivalent American version, for example *colour* and *color*
- **Date:** In US dates are written in the format *mm/dd/yyyy*, in Europe instead *dd/mm/yyyy*

Capitalization / Case-folding

Case-folding means **reducing all letters to lower case**.

This helps during queries: for example people searching for *ferrari* (lower case) are interested in the *Ferrari car*.

But sometimes the name with the capital letter is a real name (like General Motors) and it should not be put in lowercase.

To deal with this problem, an **heuristic** is to convert to lowercase words at the beginning of a sentence and all words occurring in a title that is all uppercase or in which most or all words are capitalized. Mid-sentence capitalized words are left as capitalized

Stemming and Lemmatization

For grammar reasons, in many languages the same word can be written in different ways, for example: *organize, organizes, organizing*.

A query search using one of these words should retrieve all documents containing the others.

By using **Stemming and Lemmatization** we can reduce a word into a base form

Stemming and Lemmatization

Stemming: dropping off the end of the word hoping to achieve the goal most of the time

Lemmatization: making use of a vocabulary in order to find the base form of a word, which is known as **lemma**

Example: think about the word *saw* (which is the past of *see*)

Stemming: returns *s*

am, are, is $\Rightarrow$ *be*

Lemmatization: returns *see*

car, cars, car's, cars' $\Rightarrow$ *car*

Stemming: the Porter's algorithm

An effective most common algorithm for stemming english words is the Porter's algorithm.

The aim of this algorithm is to remove the suffixes from an English word and obtain its stem in order to reduce the space required for storing the Dictionary

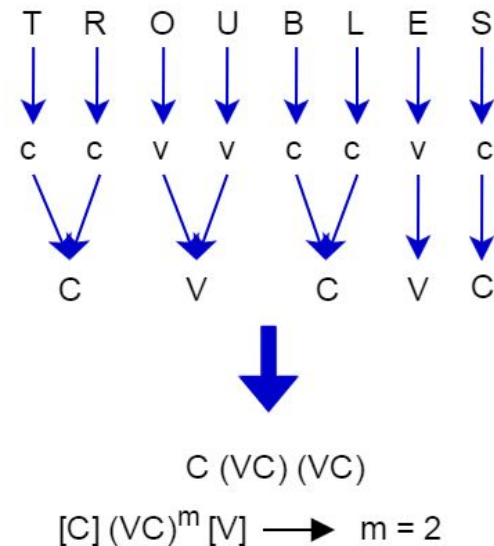
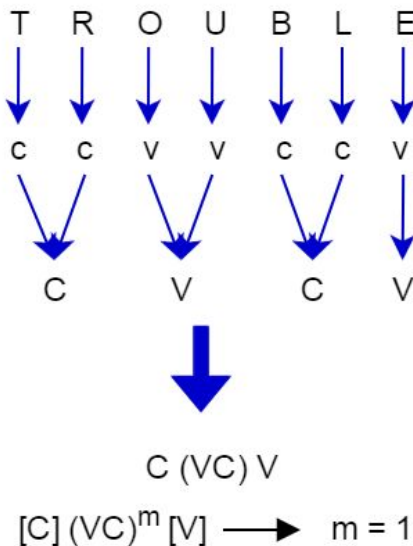
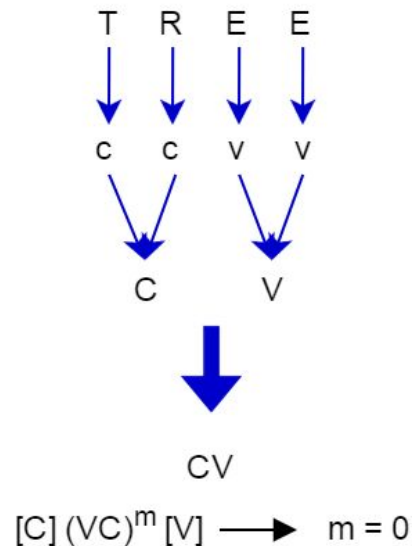
Porter's Algorithm: Consonant and Vowels

All English words can be considered as a sequence of consonant and vowel. All of them share the same pattern

$$[C](VC)^m[V]$$

m is called **measure** and represent the number of pairs Vowel-Consonant appearing in the original word.

Porter's Algorithm: Consonant and Vowels



Porter's Algorithm: Rules

This algorithm consists of **5 steps** of word reductions, applied **sequentially**.

On each step, some **rules** are applied on the word by replacing the ending part of the word according to the rule

A rule has the following form:

(condition) S1 -> S2

where condition (optional) define when the rule can be applied, S1 is the suffix in the original word, S2 is the suffix to replace S1

Porter's Algorithm: Conditions

A condition in the rule can contain:

- *S: the stem ends with S (and similarly for the other letters)
- *v*: the stem contains a vowel
- *d: the stem ends with a double consonant (e.g. -TT, -SS)
- *o: the stem ends cvc, where the second c is not W, X or Y (e.g. -WIL, -HOP)
- m>k: where m is the measure and k is a constant
- one or more previous rules with **AND, OR, NOT**

Porter's Algorithm: Step 1a

Step 1a

1. SSES	→	SS
2. IES	→	I
3. SS	→	SS
4. S	→	

For example CARESSES -> CARESS

Porter's Algorithm: Step 1b

Step 1b

1. (m>0) EED → EE
2. (*v*) ED →
3. (*v*) ING →

If the second or third of the rules in Step 1b is successful, the following is performed.

1. AT → ATE
2. BL → BLE
3. IZ → IZE
4. (*d and not (*L or *S or *Z)) → single letter
5. (m=1 and *o) → E

Porter's Algorithm: Step 1c

Step 1c

| 1. (*v*) Y → I

Porter's Algorithm: Step 2

Step 2

1. (m>0) ATIONAL	→	ATE	11. (m>0) IZATION	→	IZE
2. (m>0) TIONAL	→	TION	12. (m>0) ATION	→	ATE
3. (m>0) ENCI	→	ENCE	13. (m>0) ATOR	→	ATE
4. (m>0) ANCI	→	ANCE	14. (m>0) ALISM	→	AL
5. (m>0) IZER	→	IZE	15. (m>0) IVENESS	→	IVE
6. (m>0) ABLI	→	ABLE	16. (m>0) FULNESS	→	FUL
7. (m>0) ALLI	→	AL	17. (m>0) OUSNESS	→	OUS
8. (m>0) ENTLI	→	ENT	18. (m>0) ALITI	→	AL
9. (m>0) ELI	→	E	19. (m>0) IVITI	→	IVE
10. (m>0) OUSLI	→	OUS	20. (m>0) BILITI	→	BLE

Porter's Algorithm: Step 3

Step 3

1. (m>0) ICATE	→	IC
2. (m>0) ATIVE	→	
3. (m>0) ALIZE	→	AL
4. (m>0) ICITI	→	IC
5. (m>0) ICAL	→	IC
6. (m>0) FUL	→	
7. (m>0) NESS	→	

Porter's Algorithm: Step 4

Step 4

1. (m>1) AL →
2. (m>1) ANCE →
3. (m>1) ENCE →
4. (m>1) ER →
5. (m>1) IC →
6. (m>1) ABLE →
7. (m>1) IBLE →
8. (m>1) ANT →
9. (m>1) EMENT →
10. (m>1) MENT →

11. (m>1) ENT →
12. (m>1 and (*S or *T)) ION →
13. (m>1) OU →
14. (m>1) ISM →
15. (m>1) ATE →
16. (m>1) ITI →
17. (m>1) OUS →
18. (m>1) IVE →
19. (m>1) IZE →

Porter's Algorithm: Step 5

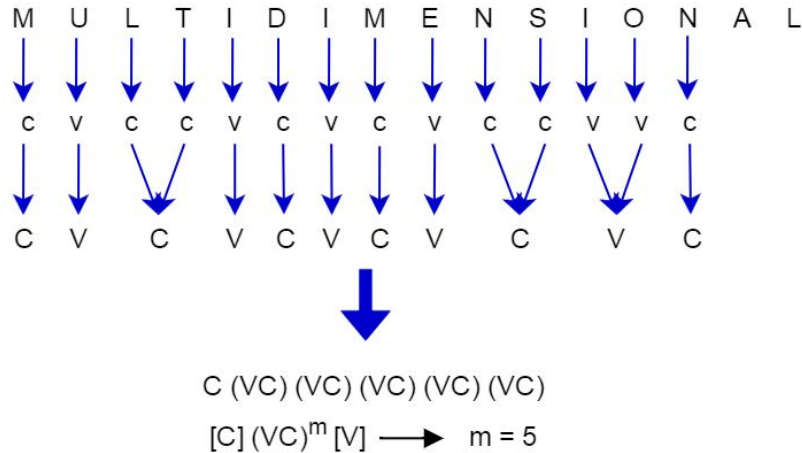
Step 5a

- 1. (m>1) E →
- 2. (m=1 and not *o) E →

Step 5b

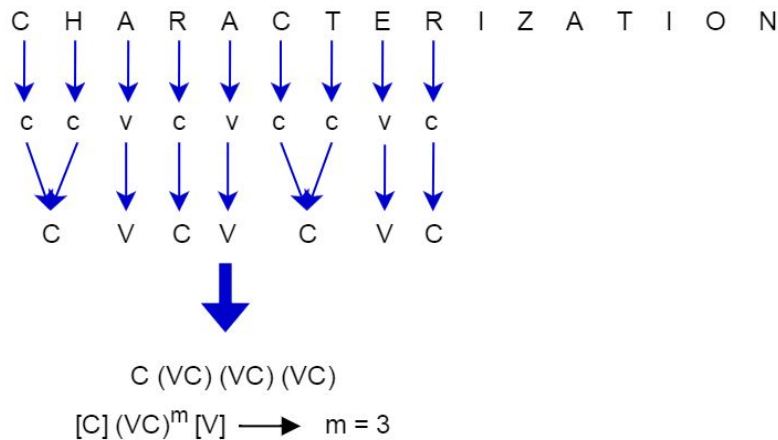
- 1. (m > 1 and *d and *L) → single letter

Porter's Algorithm: Example 1



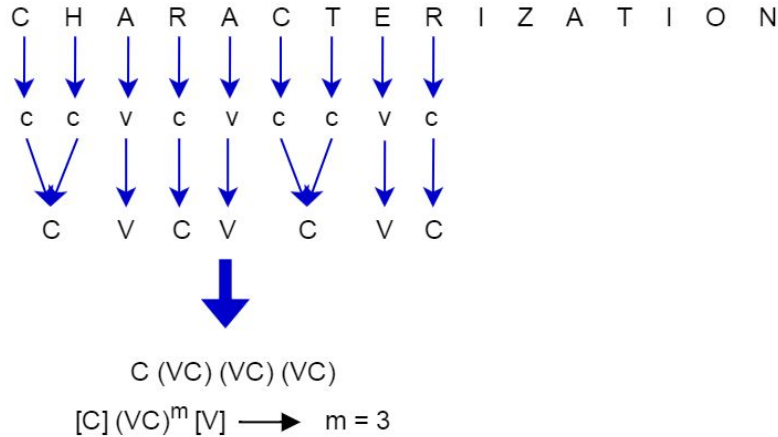
1. The suffix will not match any of the cases found in steps 1, 2 and 3.
2. Then it comes to step 4.
3. The stem of the word has $m > 1$ (since $m = 5$) and ends with "AL".
4. Hence in step 4, "AL" is deleted (replaced with null).
5. Calling step 5 will not change the stem further.
6. Finally the output will be MULTIDIMENSION.

Porter's Algorithm: Example 2



1. The suffix will not match any of the cases found in step 1.
2. So it will move to step 2.
3. The stem of the word has $m > 0$ (since $m = 3$) and ends with "IZATION".
4. Hence in step 2, "IZATION" will be replaced with "IZE".
5. Then the new stem will be CHARACTERIZE.

Porter's Algorithm: Example 2



6. Step 3 will not match any of the suffixes and hence will move to step 4.
7. Now $m > 1$ (since $m = 3$) and the stem ends with "IZE".
8. So in step 4, "IZE" will be deleted (replaced with null).
9. No change will happen to the stem in other steps.
10. Finally the output will be CHARACTER.

Stemming: other algorithms

Sample text: Such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Lovins stemmer: such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Porter stemmer: such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Paice stemmer: such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

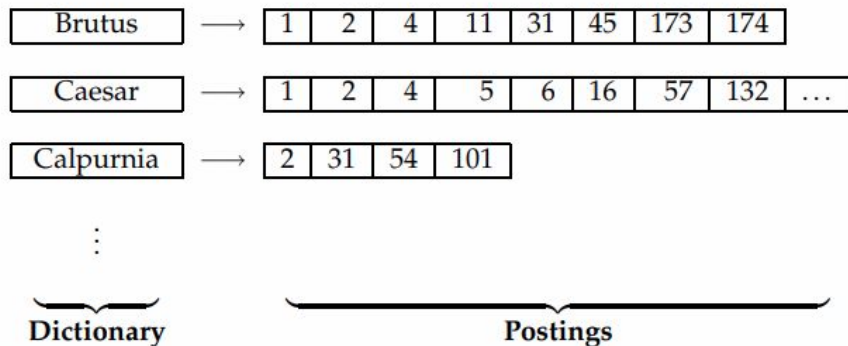
Postings List

Postings List

The Postings List is a list of all documents that are indexed with a term.

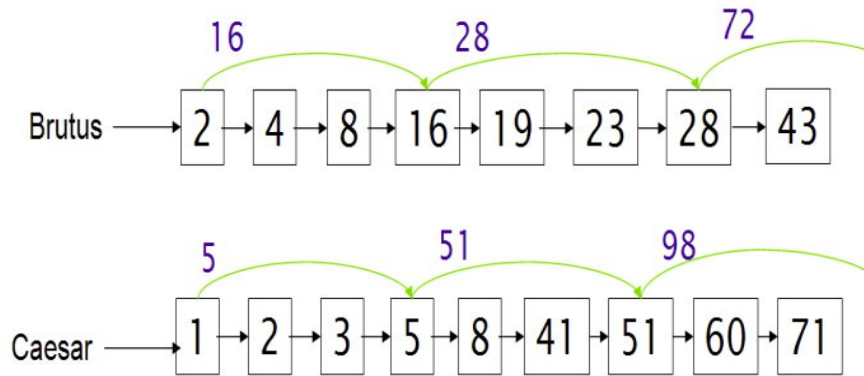
Previously, we discussed about the time for the **intersection operation**, which takes $O(n+m)$, where n and m are the length of two postings list.

Can we do better?



Skip List

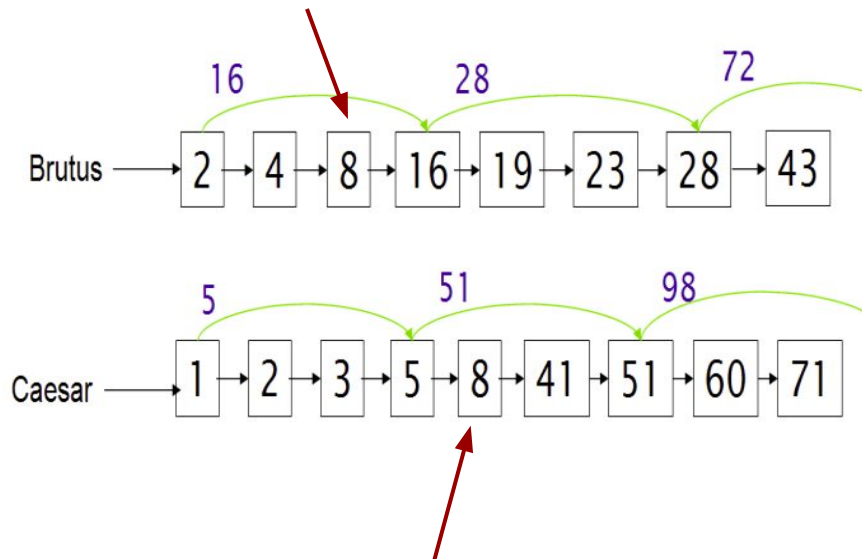
At indexing time (when index is built) we can add some **shortcuts**, that are points to other elements of the Postings Lists, used to make intersection faster (which means taking **sublinear time**)



Skip List

Let's see how skip pointers can effectively make intersection faster:

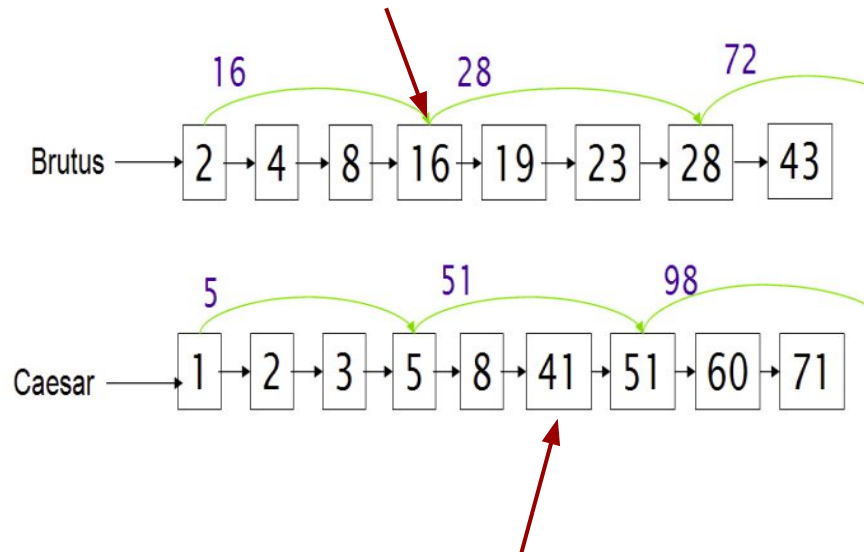
- Suppose we stepped through the lists until we have matched 8
- we move forward the pointers on both list (the first goes to 16, the second to 41)



Skip List

Let's see how skip pointers can effectively make intersection faster:

- Rather than simply moving the first pointer throughout all elements till we reach 41 (if available), we can easily jump to directly 28 (items are **ordered**)



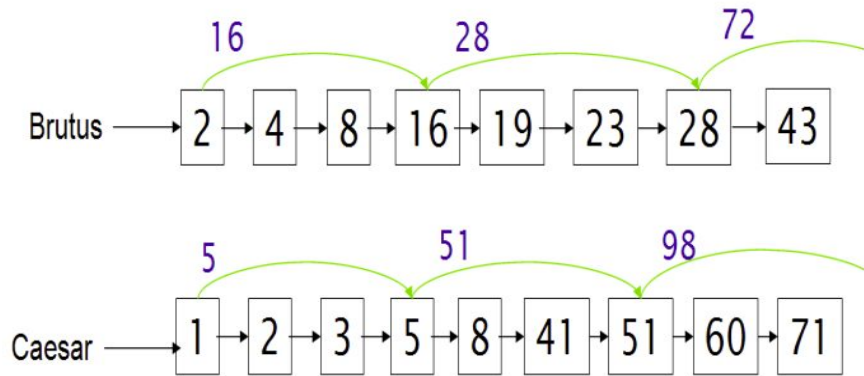
Intersection with Skip

INTERSECTWITHSKIPS(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(\text{answer}, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then if  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
9          then while  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
10             do  $p_1 \leftarrow \text{skip}(p_1)$ 
11             else  $p_1 \leftarrow \text{next}(p_1)$ 
12  else if  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
13      then while  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
14          do  $p_2 \leftarrow \text{skip}(p_2)$ 
15          else  $p_2 \leftarrow \text{next}(p_2)$ 
16  return answer
```

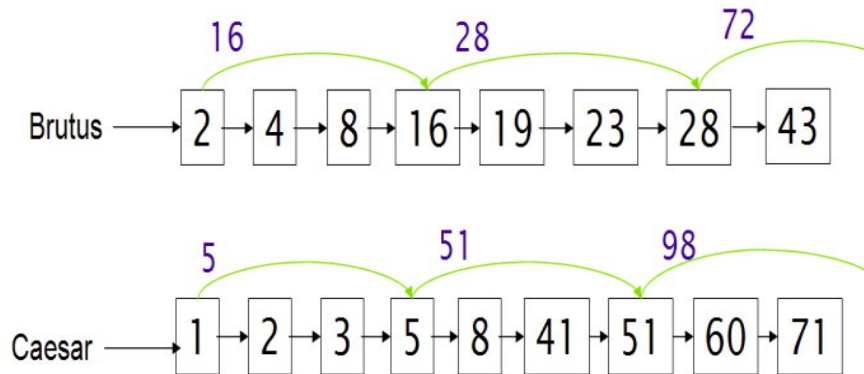

Considerations on Skip List

- Skip List only works with *AND* queries, and not for *OR* queries
- Where a Skip Pointer should be put?
 - In the example, a Skip Point is put each 4 items. Is it a good choice?
 - There is a **tradeoff**



Considerations on Skip List

- Shorter Skip Span means:
 - more Skip Pointers
 - more comparisons of pointers
 - more space to store pointers
 - more opportunities to skip



A **simple heuristic** for placing skips, which has been found to work well in practice, is that for a posting list of length P , use **$\sqrt{P}$** evenly-spaced skip pointers.

The hardness of managing Skip List

If an index **changes hardly ever**, dealing with Skip List is quite simple.

Frequent updates in the index makes management very complex because we need **to update all pointers** when a new item is added to the postings list.

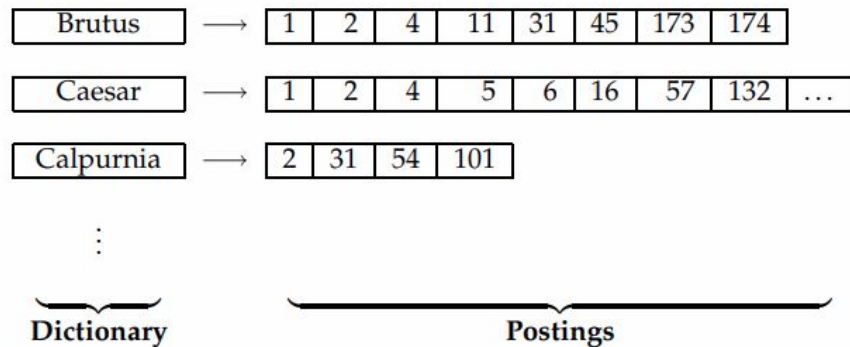
Remind: **Postings Lists are sorted** by Document ID

The problem of single terms

So far we have been thinking about the Dictionary like a **single term** structure.

This works fine but sometimes query results can be unexpected

For example:



The problem of single terms

Suppose we have the following document:

The inventor Stanford Ovshinsky never went to university

and we want to search for *Stanford University*.

Being the words *Stanford* and *University* into the document, the document is retrieved by the Search Engine, although it actually does not refer to the Standard University.

The problem of single terms

This problem has been partially solved by the modern Search Engine by using **Phrase Queries**

A Phrase Query is a sequence of words wrapped by **double quotes**. For example *"Stanford University"*.

To be able to support such a kind of Phrase Queries we need to **extends the Postings List**.

There are two approaches: **BiWord Index** and **Positional Index**

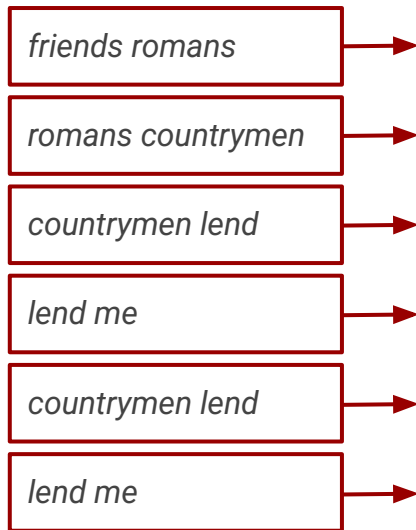
BiWord Index

The idea is very simple: every pair of consecutive terms in a document (**biword**) is considered as a phrase.

Each biword is treated as a **vocabulary term**.

Example

Friends, Romans, Countrymen, lend me your ears;



BiWord Index

Sometimes words to be considered as a Biword are not close to each other (like in *Stanford University*) but they are divided by other words (such as articles, prepositions, etc).

Consider the following example:

the abolition of the slavery or the renegotiation of the constitution

We would like to have the following biword: *abolition slavery* and *renegotiation constitution*, but those names are separated by articles and prepositions

BiWord Index

So, the idea is **to tag** all words in nouns (N) and non-nouns (X), such as articles and prepositions.

For example:

renegotiation	of	the	constitution
N	X	X	N

A biword is any string of terms of the form **NX*N**

So, *renegotiation constitution* is then used as vocabulary term.

BiWord Index

Another example:

The document containing the phrase

cost overruns on a power plant

will be indexed this the following three biwords

cost overruns

overruns power

power plant

Considerations on using BiWord Index

- searches for a single term **are not naturally handled** in a biword index
- to allow single term searches we also need **single-word index**
- **false positive** matches (non relevant documents that are retrieved) becomes very small
- storing biwords makes the **dictionary larger**
- biword index is a convenient method for implementing **proximity operators**

Phrase Index

The concept of Biword Index can be extended to **three or more words**.

If the number of the words considered like a atomic word is variable, we can talk about **Phrase Index**.

The consideration we did on Biwords must be kept into account even for Phrase Index.

Positional Index

A second approach for solving the single term problem, is the **Positional Index**.

Positional index manages Dictionary and Postings Lists in a different manner:

- vocabulary term
- term frequency in the collection
- document id
- term frequency in the document
- position of the term in the document

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$

$2, 5: \langle 1, 17, 74, 222, 255 \rangle;$

$4, 5: \langle 8, 16, 190, 429, 433 \rangle;$

$5, 2: \langle 363, 367 \rangle;$

$7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$

$4, 5: \langle 17, 191, 291, 430, 434 \rangle;$

$5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$

Positional Index

During Queries, Positional Index works in a similar manner of general:

We scan the inverted index starting from terms less frequent. In the **Intersection Operation**, rather than simply checking that both terms are in a document, we also need to check that their positions of appearance in the document are **compatible with the phrase** query being evaluated.

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$
 $2, 5: \langle 1, 17, 74, 222, 255 \rangle;$
 $4, 5: \langle 8, 16, 190, 429, 433 \rangle;$
 $5, 2: \langle 363, 367 \rangle;$
 $7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$
 $4, 5: \langle 17, 191, 291, 430, 434 \rangle;$
 $5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$

Positional Index

An example: suppose we want to search for *to be or not to be*. We look for the terms *to*, *be*, *not*, or in the posting list.

Now, let's consider the intersection between *to* and *be*.

Documents 1, 4, 5 contain both terms.

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$
 $2, 5: \langle 1, 17, 74, 222, 255 \rangle;$
 $4, 5: \langle 8, 16, 190, 429, 433 \rangle;$
 $5, 2: \langle 363, 367 \rangle;$
 $7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$
 $4, 5: \langle 17, 191, 291, 430, 434 \rangle;$
 $5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$

Positional Index

The aim of the Positional Index is to give a **measure of relevance**: documents having both terms closer are more relevant to others.

Then we look the posting list of *be* to see if there is an index **one higher** than a position of *to*

to, 993427:

⟨ 1, 6: ⟨7, 18, 33, 72, 86, 231⟩;

2, 5: ⟨1, 17, 74, 222, 255⟩;

4, 5: ⟨8, 16, 190, 429, 433⟩;

5, 2: ⟨363, 367⟩;

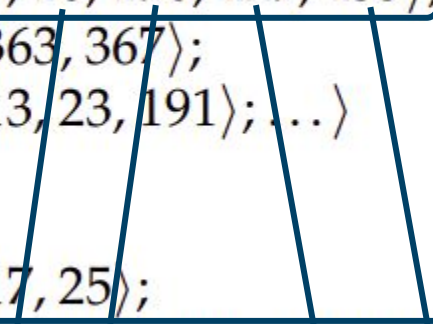
7, 3: ⟨13, 23, 191⟩; ... ⟩

be, 178239:

⟨ 1, 2: ⟨17, 25⟩;

4, 5: ⟨17, 191, 291, 430, 434⟩;

5, 3: ⟨14, 19, 101⟩; ... ⟩



Positional Index

In this example, we used $K = 1$ (we looked for an instance of *be* having an index position **one higher** of *to*)

We can also generalize the algorithm for any K .

The following pseudocode shows the Positional Intersection with a generic K

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$

$2, 5: \langle 1, 17, 74, 222, 255 \rangle;$

$4, 5: \langle 8, 16, 190, 429, 433 \rangle;$

$5, 2: \langle 363, 367 \rangle;$

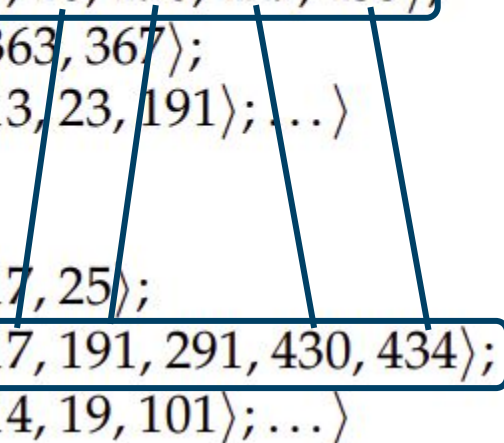
$7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$

$4, 5: \langle 17, 191, 291, 430, 434 \rangle;$

$5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$



Positional Index

```
POSITIONALINTERSECT( $p_1, p_2, k$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $l \leftarrow \langle \rangle$ 
5           $pp_1 \leftarrow \text{positions}(p_1)$ 
6           $pp_2 \leftarrow \text{positions}(p_2)$ 
7          while  $pp_1 \neq \text{NIL}$ 
8          do while  $pp_2 \neq \text{NIL}$ 
9              do if  $|\text{pos}(pp_1) - \text{pos}(pp_2)| \leq k$ 
10                  then  $\text{ADD}(l, \text{pos}(pp_2))$ 
11                  else if  $\text{pos}(pp_2) > \text{pos}(pp_1)$ 
12                      then break
13                       $pp_2 \leftarrow \text{next}(pp_2)$ 
14                  while  $l \neq \langle \rangle$  and  $|l[0] - \text{pos}(pp_1)| > k$ 
15                      do  $\text{DELETE}(l[0])$ 
16                      for each  $ps \in l$ 
17                          do  $\text{ADD}(answer, \langle \text{docID}(p_1), \text{pos}(pp_1), ps \rangle)$ 
18                           $pp_1 \leftarrow \text{next}(pp_1)$ 
19                       $p_1 \leftarrow \text{next}(p_1)$ 
20                       $p_2 \leftarrow \text{next}(p_2)$ 
21                  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
22                      then  $p_1 \leftarrow \text{next}(p_1)$ 
23                      else  $p_2 \leftarrow \text{next}(p_2)$ 
24  return  $answer$ 
```

Consideration on Positional Index

- Using Positional Index makes the **size of the inverted index larger**
 - the index size depends on the average document size
- The asymptotic complexity of a postings **intersection operation** changes
 - from **$O(N)$ to $O(T)$** where N is the number of documents, while T is the number of tokens
- It does not work with BiWord indexes
- Positional index is another convenient method for implementing **proximity operators**

Combining schemas

The strategies of biword indexes and positional indexes can be fruitfully **combined**.

For example, for certain queries we can use Phrase Index or Biword Index, for others instead we use Positional Index.

Terms to be included in a Phrase Index are those that are known to be common **based on recent querying behavior**.